

Chapter 3. Cryptographic Primitives

This chapter introduces the various cryptographic primitives, algorithms and protocol building blocks used in this protocol. It introduces for each of them a functional abstraction that can be referred to in the other chapters of this specification. This chapter also maps those cryptographic primitives to specific instances with the corresponding appropriate informative or normative references. Wherever relevant, it also gives necessary or relevant information about the use of these mappings in a specific context to achieve a compliant implementation.

Given a [version](#) of the [Message Format](#), the cryptographic primitives are mapped to specific instances. There is no cryptosuite negotiation in this protocol: one [version](#) of the [Message Format](#) has one cryptosuite as defined in this chapter.

Each section defines cryptographic primitives generically, together with concrete mappings to specific instances of these cryptographic primitives for [version 1.0](#) of the [Message Format](#). This chapter can also be used as guidance about which cryptographic primitives need to be supported by a device, but it must be noted that not all devices will have to support all of them. For example, a device may not require the `Crypto_PBKDF()` primitive, as values based on this operation could in some instances be precomputed and stored during the manufacturing process of the device. The proposed functional mapping in this chapter is normative with respect to the values computed by the functions but informative with respect to the way the functions are interfaced within implementations. For example, a function returning both a `boolean` to indicate success and a value if the operation is successful could also be implemented using exception mechanisms instead of returning a `boolean`.

It must also be noted that not all cryptographic primitives are exposed to the other parts of the specification. For example, the `Crypto_TRNG()` primitive SHALL NOT be called outside of the `Crypto_DRBG()` implementation.

The cryptographic primitives discussed below operate on data local to the host. Where more complex data types are present and their external representation is applicable, the chapter notes the details of the encoding. Simple multi-byte data types without any additional context are assumed to be in host byte order when they are used internally to a procedure, unless otherwise stated.

All octet strings are presented with first octet having index 0, and presented from left to right for indices 0 through N-1 for an octet string of length N.

3.1. Deterministic Random Bit Generator (DRBG)

This protocol relies on random numbers for many security purposes. For example, random numbers are used in generating secret keys, counters, cryptographic signature generation random secrets, etc. Those random numbers SHALL be generated using the `Crypto_DRBG()` function.